Phase 4, days 91 to 180: Enable

EY: Talent Data Strategy

C8 Copilot and AI in Fabric

C8-L1 Copilot in Fabric, where it runs and what it costs

Confirm Copilot is enabled on a paid capacity, record the release status of each experience, and budget its CU use.

Copilot in Fabric capacity requirement	Paid F2+ or P1+ in a supported region, with tenant settings enabled. Not after a trial expires.
Copilot release status	Per experience. Notebooks and Warehouse SQL preview. Data agents, Data Factory and Power BI GA (September 2026).
Copilot token rates per 1,000 tokens	Input 100, cached input 10, output 400 CU seconds. Background job, smoothed over 24 hours.
Copilot output	Sometimes inaccurate. A person able to judge it reviews it before use.

C8-L2 Data agents, questions in plain language

Narrow the schema, define your terms, add validated examples, and measure accuracy against known answers.

Data agent query languages	SQL for lakehouse and warehouse, DAX for semantic models, KQL for KQL databases. Up to five sources.
Data agent security	Read-only queries only. Runs with the asking user's permissions.
Example queries	Question and query pairs, validated against the schema. Not supported for semantic models.
Measure a data agent	Ground-truth questions with known answers, rerun after every change, accuracy recorded.

C8-L3 Verify what Copilot writes

Read it, run it on data with a hand-worked answer, compare it with a second method, and keep those checks as tests.

A Copilot query ran without error. What does that prove?	Syntax parses and names exist. Nothing about correctness.
Four checks for Copilot output	Read every line. Run on known data. Compare with a second method. Keep the checks as tests.
Average of averages	Weights small groups like large ones. Divide total by total instead.
Known data for verification	Five to ten hand-built rows with a null, a zero denominator, a duplicate key and an empty group.

A7 Observability

A7-L1 Structured logging with the logging module, not print

Every line says what, where, which run and which wave. Failures keep their traceback and still fail the job.

```
base = logging.getLogger(__name__)
log = logging.LoggerAdapter(base, {"run_id": run_id, "wave_id": wave_id})
log.info("merged rows=%d", n)
# INFO pipelines.silver run_id=7f3c wave_id=W14 merged rows=1832
```

Create a module logger	`logger = logging.getLogger(__name__)`
Default level threshold with no configuration	WARNING. log.info(...) prints nothing until the entry point sets a level.
Attach run_id and wave_id to every line	logging.LoggerAdapter(logger, {"run_id": ..., "wave_id": ...})
Inside except	log.exception("...") for ERROR plus traceback, then a bare raise.

A7-L2 A pipeline run log table with an explicit schema

The strong record builds on every run, success or failure, and its types never depend on the values in one row.

Why does the reference log_pipeline_run fail on success?	error_message=None is inferred with no type: CANNOT_DETERMINE_TYPE. Declare a StructType.
UTC timestamp in modern Python	datetime.now(timezone.utc). utcnow() is deprecated from 3.12 and naive.
Which ruff rule catches the missing uuid import?	F821, undefined name.
Where do you write the run record so failures are logged too?	In a finally block, after setting status in try and except.

A7-L3 Alert on failures, and on the silence that hides them

Job notifications catch runs that fail. Scheduled queries catch runs that do not happen, or that succeed at doing nothing.

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

Why must an except block re-raise?	Otherwise the run succeeds and on_failure notifications never fire.
Job-level versus task-level failure notifications	Job-level notifications are not sent for retried task failures. Use task-level ones to hear about every failed attempt.
How do you detect a pipeline that stopped running?	A scheduled query: no SUCCESS in the run log within the expected window.
Where should alerts go?	A group address or channel, with pipeline, wave, run ID and a link.

B22 Evaluating LLM systems

B22-L1 Evaluate retrieval, recall at k and MRR on a gold set

Score retrieval apart from generation, on a versioned gold set, with recall at k and MRR, and compare settings on the same questions.

```python
def recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float:
    return len(set(ranked[:k]) & relevant) / len(relevant)


def reciprocal_rank(ranked: list[str], relevant: set[str]) -> float:
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0
```

Recall at k	len(set(ranked[:k]) & relevant) / len(relevant). With one relevant section per question, its mean is the hit rate.
Mean reciprocal rank	Mean over questions of 1 / rank of the first relevant result, ranks from 1, or 0 when none appears.
Gold set rules	Real questions, relevance marked by section, versioned, and every failed question added.
Repeated sections in a ranking	list(dict.fromkeys(section_ids)) keeps each at its first position.

B22-L2 Evaluate answers, exact checks, token F1 and LLM judges

Exact checks first, token F1 for trends, and a validated, structured judge for free text, run in both orders for pairs.

Order of answer checks	Exact checks first, then token F1 for trends, then a validated LLM judge for free text.
Token F1	Shared words with Counter &; p = shared / reply words, r = shared / reference words, F1 = 2pr / (p + r).
LLM judge biases (Zheng et al., NeurIPS 2023)	Position, verbosity and self-enhancement bias. Swap the order for pairs and call disagreements ties.
Before trusting a judge	Compare it with human grades on a sample, criterion by criterion, and re-check after any prompt or model change.

B22-L3 Regression evals and release gates for prompt changes

Compare per item, repeat noisy items, write the gate first, and never let a higher average hide a broken critical item.

```python
fixed = sorted(q for q in baseline if not baseline[q] and candidate[q])
broken = sorted(q for q in baseline if baseline[q] and not candidate[q])
```

Fixed and broken items	Fixed: fail in the baseline, pass in the candidate. Broken: the reverse. Read every broken item.
Majority over repeated runs	Pass when sum(runs) > len(runs) / 2. Unstable when 0 < sum(runs) < len(runs).
A release gate	Written before the run: every critical item passes, at most N items break, and the pass rate drops by no more than a set share.

What to log for each eval run	Model, prompt version, eval set version, verdicts per item and token counts.